Comparing Apache Spark, Dask, and RAPIDS
on Batch and Iterative Workloads

1 Objective and implementation

We implemented comparable data-processing pipelines in **Apache Spark** (DataFrame API), **Dask** (CPU DataFrames), and **RAPIDS/Dask-cuDF** (GPU DataFrames). The objective was to evaluate how execution model and workload characteristics affect end-to-end runtime, CPU parallel scaling, and memory use. The submission includes implementations, data preparation, benchmark runners, validation, CSV results, and plotting scripts.

2 Dataset and workloads

We used public **NYC TLC Yellow Taxi** trip records from January–March 2024, stored as Parquet, together with the taxi-zone lookup CSV. The original monthly files are available at `https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page`. A shared preparation script creates four subsets containing **100K, 500K, 1M, and 2M** trips; these prepared files are omitted from the submission because of their size.

All frameworks implement three workloads: (1) **aggregation**, grouping by pickup zone and hour to compute counts and fare/distance statistics; (2) **join**, enriching trips with zone information and joining derived zone statistics; and (3) **K-means**, running ten Lloyd iterations with five centroids and fixed initialization on selected numeric trip features. The same prepared input files are used across frameworks.

3 Experimental methodology

Dask and Spark were tested with **1, 2, 4, and 8 CPU workers**, using 16 partitions; RAPIDS ran on a **single NVIDIA Tesla T4** with 16 partitions. Each workload/size/configuration had **one untimed warmup and three timed repetitions**. We report the median wall-clock time; 144 Dask, 144 Spark, and 36 RAPIDS measurements completed successfully (324 total). CPU experiments used Python 3.14.8; Spark used Java 17. RAPIDS used Python 3.13.16, cuDF 26.06, and a Tesla T4 (15 GiB). Exact CPU processor/RAM details were not captured in the supplied configuration records.

The runners recorded process resident memory (RSS); RAPIDS additionally sampled GPU device memory. These are different metrics and are *not* directly interchangeable. We compared output row counts and checksums across frameworks: all recorded configurations agreed within floating-point tolerance. Checksums are useful sanity checks but do not establish complete equivalence of intermediate execution plans.

4 Results

Figure 1 shows that aggregation and join times change relatively little with increasing dataset size, especially for RAPIDS; fixed execution and scheduling costs are plausible contributors at these scales. K-means grows more markedly and shows the strongest benefit from GPU execution. At 2M rows, the medians in Table 1 show that **single-worker Dask beats RAPIDS on aggregation and join**, whereas **RAPIDS is 3.40× faster than single-worker Dask and 47.49× faster than single-worker Spark on K-means**. These ratios compare observed systems, not equivalent CPU/GPU hardware budgets.

Table 1: Median seconds at 2M rows (lower is better).

Workload	Dask (1)	Spark (1)	RAPIDS (1 GPU)
Aggregation	**0.964**	2.234	1.200
Join	**1.909**	3.018	2.396
K-means	10.403	145.320	**3.060**

Figure 2 illustrates diminishing CPU scaling returns. For example, Dask K-means improves from 10.40 s with one worker to 4.37 s with four, then slows to 4.78 s with eight. Spark K-means improves from 145.32 s to 46.91 s at four workers, but rises to 53.90 s at eight. Scheduling, synchronization, and data movement are possible causes; the experiments did not separately profile these costs. GPU memory sampling and CPU RSS also reflect caching and allocator behavior, so memory deltas should not be treated as exact per-operation allocation sizes.

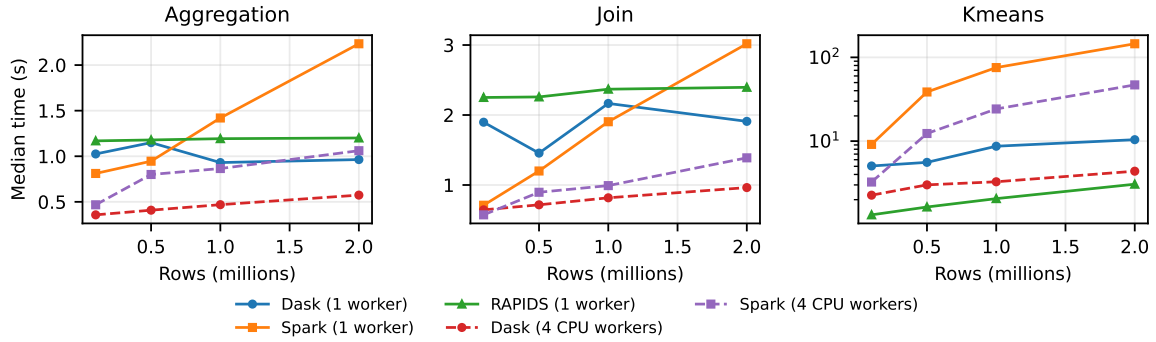


Figure 1: Median runtime versus dataset size. Solid: one worker; dashed: four CPU workers. K-means uses a logarithmic vertical axis.

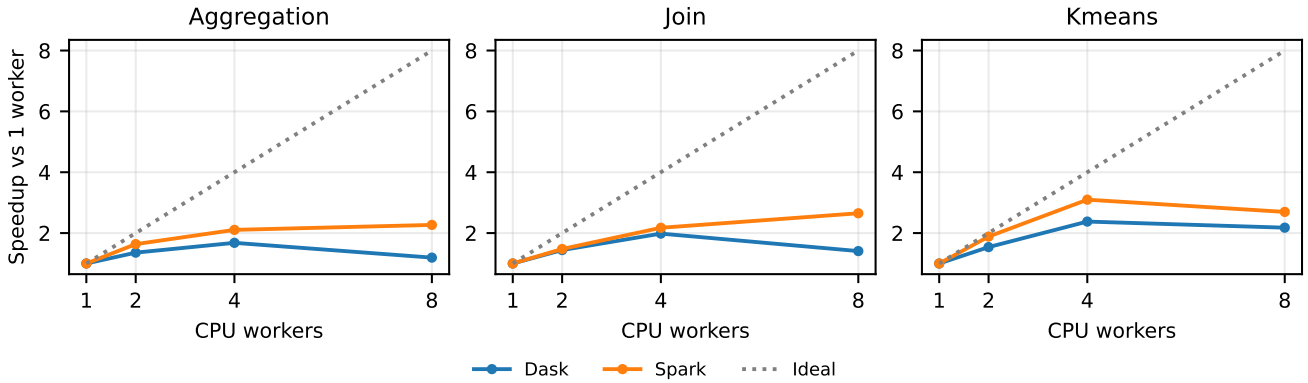


Figure 2: CPU speedup at 2M rows relative to each framework's one-worker runtime. The dotted line represents ideal linear scaling. RAPIDS is excluded because only one GPU was available.

5 Limitations and conclusions

The GPU and CPU runs used **different machines and software environments**; thus performance ratios are not hardware-normalized acceleration factors. Three repetitions permit descriptive variability analysis but limited statistical inference. The join checksum could be insensitive to some intermediate join operations, and K-means centroid sums do not prove identical cluster assignments. Finally, no direct shuffle-byte or GPU-utilization trace was retained.

Overall, **no framework wins every workload**: CPU Dask is competitive for relatively lightweight batch operations, while RAPIDS provides a substantial observed advantage for iterative K-means at the largest tested size. The results suggest choosing an execution engine based on workload and overhead rather than GPU availability alone.

6 Reproducing the experiments

From the repository root, download the three 2024 Yellow Taxi Parquet files into `data/raw/` and run `python common/prepare_data.py -months 2024-01 2024-02 2024-03 -sizes 100000 500000 1000000 2000000`. Install the CPU requirements in separate environments (`requirements.txt` for Dask; `requirements-spark.txt` and Java 17 for Spark), then run `bash run_dask_final.sh` and `bash run_spark_final.sh`. On a CUDA-enabled machine with compatible RAPIDS/cuDF installed, run `bash run_rapids_smoke.sh` followed by `bash run_rapids_final.sh`. Outputs are stored under `results/`; analysis scripts are in `analysis/`. The README documents environment setup and file locations.